

Lab-two: Raft

论文

☒ Raft

Raft

这篇论文是斯坦福大学Diego Ongaro和John Ousterhout的经典研究，提出了共识算法**Raft**，核心是打造比Paxos更易理解、且性能与正确性相当的共识机制，用于管理分布式系统中的复制日志，支撑高可用的复制状态机，以下是核心内容总结：

一、研究背景与动机

1. **Paxos的问题**：作为分布式共识的经典算法，Paxos虽正确高效，但存在两大核心缺陷——**理解难度极高**（单决议Paxos设计晦涩，多决议扩展更复杂，即使资深研究者也难以掌握）；**工程实现性差**（多决议Paxos无统一标准实现，核心的对等架构与实际系统的领导者模式不符，落地时需大幅修改，易出错）。
2. **研究目标**：设计新的共识算法，将**可理解性**作为首要目标，同时保证正确性、可用性、效率，为分布式系统实现和教学提供更好的基础。

二、Raft的核心设计原则

为提升可理解性，Raft采用两大核心设计策略：

1. **问题分解**：将共识问题拆分为**领导者选举**、**日志复制**、**安全性保障**、**集群成员变更**四个独立子问题，各部分可单独设计、解释和理解。
2. **状态空间简化**：减少系统的非确定性，限制日志不一致的情况（如日志无空洞）；仅在必要时引入随机性（如领导者选举的随机超时），以简化逻辑、避免复杂的边界情况。

三、Raft共识算法核心机制

Raft是**领导者主导**的共识算法，集群中节点仅有**领导者**、**跟随者**、**候选者**三种状态，通过**任期（Term）**作为逻辑时钟解决分布式时钟同步问题，保证同一任期至多一个领导者，核心由四部分组成：

1. 领导者选举

- 触发条件：跟随者在**选举超时**内未收到领导者的心跳（空AppendEntries RPC），则转为候选者，开启选举。

- 选举过程：候选者自增任期、投自己一票，并向所有节点发送RequestVote RPC；获得**多数节点投票**则成为领导者，立即发送心跳维持权威。
- 关键优化：采用**随机选举超时**（如150-300ms），避免投票分裂，快速解决选举冲突；投票时会检查候选者日志的“最新性”（比较最后日志的任期和索引），保证新领导者包含所有已提交的日志条目。

2. 日志复制

- 核心流程：领导者接收客户端命令，追加到本地日志，并行向所有跟随者发送AppendEntries RPC复制日志；当日志被**多数节点复制**后，领导者将其标记为**已提交**，并通知跟随者应用到状态机，同时向客户端返回结果。
- 日志一致性保障：日志条目包含**任期号**和**索引**，AppendEntries包含前一条目的索引和任期，跟随者验证不通过则拒绝复制；领导者通过维护 `nextIndex`（向跟随者发送下一条日志的索引）逐步修正跟随者的不一致日志，强制跟随者日志与领导者保持一致，且领导者日志**仅追加、不修改/删除**。
- 关键特性：日志匹配性——若两个日志在某索引和任期的条目相同，则其之前的所有条目完全一致，保证日志的连贯性。

3. 安全性保障

- 核心安全属性：**状态机安全**——若某节点将某索引的日志应用到状态机，其他节点绝不会对同一索引应用不同日志。
- 核心保障机制：**领导者完整性**——已提交的日志条目会出现在所有更高任期的领导者日志中，通过投票时的日志最新性检查实现；仅允许领导者通过**当前任期的日志提交**间接确认旧任期日志的提交，避免旧日志被覆盖的边界问题。
- 异常处理：跟随者/候选者崩溃后，领导者无限重试RPC，且RPC具有**幂等性**，崩溃节点重启后可正常恢复。

4. 集群成员变更

- 核心问题：直接切换集群配置会导致出现两个独立多数派，引发双领导者问题，破坏安全性。
- 解决方案：**联合共识（Joint Consensus）** 两阶段切换——先切换到包含新旧配置的过渡配置，日志复制到所有节点，且选举/提交需同时获得新旧配置的多数派同意；过渡配置提交后，再切换到新配置，保证切换过程中集群可正常处理请求，无安全风险。
- 补充优化：新节点先以**非投票成员**加入集群，同步完日志后再参与投票，避免可用性下降；移除的节点会被忽略投票请求，防止其干扰集群。

四、Raft的扩展机制

1. 日志压缩

- 核心问题：日志无限增长会导致存储耗尽、重启回放时间过长，需丢弃过期日志。
- 实现方式：采用**快照（Snapshot）**——节点独立将已提交日志对应的状态机完整状态写入快照，包含最后提交条目的索引和任期，快照生成后可删除对应日志；领导者通过InstallSnapshot RPC向落后的跟随者发送快照，快速同步状态。
- 优化：使用写时复制（Copy-on-Write），避免快照生成阻塞正常业务。

2. 客户端交互

- 领导者发现：客户端随机连接节点，非领导者会重定向到领导者；领导者崩溃则客户端超时后重新随机连接。
- 线性一致性保障：客户端为每个命令分配唯一序列号，状态机记录已处理的序列号，避免命令重复执行；只读操作需领导者先向多数节点发送心跳，确认自身仍为有效领导者，避免返回过期数据。

五、Raft的实现与评估

1. **工程实现**：作者基于C++实现Raft，核心代码仅约2000行，已开源；同时存在大量第三方开源实现，被多家公司部署使用。
2. **可理解性评估**：对斯坦福和伯克利的43名本科生/研究生开展用户研究，对比Raft和Paxos的学习效果，结果显示：
 - 学生在Raft测试中的平均分显著高于Paxos，33名学生对Raft的问题回答表现更好；
 - 绝大多数学生认为Raft更易实现和讲解，即使实验设计对Paxos更有利（如部分学生有Paxos基础、Paxos讲解视频更长）。
3. **正确性验证**：用TLA+形式化语言编写Raft规范，机械证明了**日志完整性**；同时完成了**状态机安全**的详细非形式化证明，保证算法的数学正确性。
4. **性能评估**：Raft的性能与Paxos相当，正常情况下日志复制仅需**一轮RPC到多数节点**，支持批处理和流水线优化提升吞吐量；领导者选举的性能测试显示：
 - 随机选举超时可有效避免投票分裂，150-300ms的超时设置可保证低宕机时间（中位数约287ms）；
 - 更小的超时（如12-24ms）可进一步降低选举时间，但易导致不必要的领导者切换，降低系统可用性。

六、相关工作与对比

1. **与Paxos对比**：Raft的**强领导者**设计是核心差异——Paxos将领导者选举作为性能优化，核心是对等架构，而Raft将领导者选举作为共识的核心环节，所有核心逻辑由领导者主导，大幅减少机制复杂度；Raft的消息类型仅4种（2种RPC请求+响应），远少于Paxos的扩展实现。

2. **与Viewstamped Replication (VR)、ZooKeeper (Zab) 对比：**三者均为领导者主导，但Raft进一步简化非领导者逻辑（日志仅从领导者流向跟随者，无反向传输），消息类型更少，实现更简单；ZooKeeper的实现实际与Raft更接近，但理论描述更复杂。
3. **与Egalitarian Paxos (EPaxos) 对比：**EPaxos采用无领导者设计，在部分场景（如WAN、命令可交换）下性能更高，但复杂度大幅提升，而Raft以可理解性为代价，放弃了这类小众优化。
4. **集群成员变更对比：**Raft的联合共识相比VR、SMART的方案，无需暂停正常请求，机制更简单，且充分复用了Raft本身的共识逻辑。

七、结论与贡献

1. **核心贡献：**提出了Raft共识算法，首次将**可理解性**作为分布式算法的首要设计目标，通过问题分解和状态空间简化，打造了比Paxos更易学习、实现和扩展的共识机制，且在正确性、效率、可用性上与Paxos相当。
2. **实践价值：**Raft为分布式系统的高可用实现提供了简洁、可靠的基础，已被广泛应用于工业界和学术界，成为共识算法的主流选择之一；同时为分布式算法的设计提供了新的思路——可理解性是保证工程落地正确性的关键。
3. **教学价值：**Raft的简洁性使其成为分布式共识教学的优选算法，解决了Paxos难以教学的问题，帮助学习者快速建立分布式共识的直觉。

此外，论文还强调了算法可理解性的重要性：即使算法在理论上正确高效，若开发者无法理解其核心逻辑，落地时也易偏离设计、引入错误，因此可理解性应与正确性、效率并列成为算法设计的核心目标。

研究背景与动机

- Paxos的核心缺陷
 - 理解难度极高：单决议设计晦涩，多决议扩展复杂
 - 工程实现性差：无统一多决议实现，对等架构不符合实际
- 研究核心目标
 - 首要：提升可理解性
 - 基础：保证正确性、可用性、效率
 - 目的：为系统实现和教学提供更好基础

核心设计原则

- 问题分解：拆分为领导者选举、日志复制、安全保障、成员变更四大独立子问题
- 状态空间简化
 - 减少非确定性，限制日志不一致情况
 - 必要时引入随机性（如随机选举超时）简化逻辑

Raft核心机制

- 基础设定
 - 节点状态：领导者、跟随者、候选者
 - 逻辑时钟：任期（Term），同一任期至多一个领导者
 - 核心RPC：RequestVote、AppendEntries
- 领导者选举
 - 触发：跟随者选举超时未收到心跳，转为候选者
 - 过程：自增任期→自投票→发送RPC→获多数投票成为领导者
 - 关键：随机选举超时防投票分裂；投票检查日志最新性
- 日志复制
 - 流程：接收客户端命令→追加本地日志→复制到跟随者→多数复制后标记提交→应用到状态机
 - 一致性保障：日志含任期+索引，AppendEntries验证前序条目；领导者维护nextIndex修正不一致日志
 - 特性：领导者日志仅追加；日志匹配性（同索引任期则前序完全一致）
- 安全性保障
 - 核心安全属性：状态机安全（同索引无不同日志应用）
 - 核心机制：领导者完整性（已提交日志出现在更高任期领导者日志）
 - 提交规则：仅通过当前任期日志提交间接确认旧任期日志
 - 异常处理：RPC幂等性，领导者无限重试应对节点崩溃
- 集群成员变更
 - 核心问题：直接切换易出现双领导者，破坏安全
 - 解决方案：联合共识两阶段切换
 - 阶段1：过渡配置（含新旧），需双多数派同意选举/提交
 - 阶段2：过渡配置提交后，切换至新配置
 - 补充优化：新节点先非投票同步日志；忽略移除节点的投票请求

Raft扩展机制

- 日志压缩
 - 问题：日志无限增长导致存储/回放问题
 - 实现：快照（节点独立生成，含最后提交条目信息，生成后删对应日志）
 - 同步：领导者通过InstallSnapshot RPC向落后节点发快照
 - 优化：写时复制避免快照阻塞业务
- 客户端交互
 - 领导者发现：客户端随机连接，非领导者重定向；超时则重新连接
 - 线性一致性保障
 - 唯一序列号防命令重复执行
 - 只读操作：领导者先心跳确认自身有效性，避免返回过期数据

实现与评估

- 工程实现：C++核心代码约2000行，开源；多第三方实现，工业界部署
- 可理解性评估
 - 实验对象：斯坦福、伯克利43名本/研究生
 - 实验结果：Raft测试平均分更高；多数学生认为Raft更易实现、讲解
- 正确性验证：TLA+形式化规范，机械证明日志完整性；非形式化证明状态机安全
- 性能评估
 - 基础性能：与Paxos相当，正常日志复制仅一轮RPC到多数节点
 - 选举性能：150-300ms超时低宕机；更小超时易导致领导者频繁切换
 - 优化支持：兼容批处理、流水线提升吞吐量

相关工作与对比

- 与Paxos：核心差异为强领导者设计，Raft消息类型更少（4种），机制更简单
- 与VR/Zab：均为领导者主导，Raft简化非领导者逻辑，日志仅单向传输
- 与EPaxos：EPaxos无领导者，部分场景性能更高，但复杂度远高于Raft
- 成员变更对比：Raft联合共识无需暂停请求，机制更简单，复用自身共识逻辑

结论与贡献

- 核心贡献：提出Raft算法，首次将可理解性作为首要设计目标，性能与Paxos相当
- 实践价值：为分布式高可用实现提供简洁基础，成为共识算法主流选择
- 教学价值：替代Paxos成为分布式共识教学优选，帮助建立直觉
- 设计启示：可理解性与正确性、效率并列，是工程落地正确性的关键

State

Persistent state on all servers:

(Updated on stable storage before responding to RPCs)

currentTerm	latest term server has seen (initialized to 0 on first boot, increases monotonically)
votedFor	candidateId that received vote in current term (or null if none)
log[]	log entries; each entry contains command for state machine, and term when entry was received by leader (first index is 1)

Volatile state on all servers:

commitIndex	index of highest log entry known to be committed (initialized to 0, increases monotonically)
lastApplied	index of highest log entry applied to state machine (initialized to 0, increases monotonically)

Volatile state on leaders:

(Reinitialized after election)

nextIndex[]	for each server, index of the next log entry to send to that server (initialized to leader last log index + 1)
matchIndex[]	for each server, index of highest log entry known to be replicated on server (initialized to 0, increases monotonically)

AppendEntries RPC

Invoked by leader to replicate log entries (§5.3); also used as heartbeat (§5.2).

Arguments:

term	leader's term
leaderId	so follower can redirect clients
prevLogIndex	index of log entry immediately preceding new ones
prevLogTerm	term of prevLogIndex entry
entries[]	log entries to store (empty for heartbeat; may send more than one for efficiency)
leaderCommit	leader's commitIndex

Results:

term	currentTerm, for leader to update itself
success	true if follower contained entry matching prevLogIndex and prevLogTerm

Receiver implementation:

1. Reply false if term < currentTerm (§5.1)
2. Reply false if log doesn't contain an entry at prevLogIndex whose term matches prevLogTerm (§5.3)
3. If an existing entry conflicts with a new one (same index but different terms), delete the existing entry and all that follow it (§5.3)
4. Append any new entries not already in the log
5. If leaderCommit > commitIndex, set commitIndex = min(leaderCommit, index of last new entry)

RequestVote RPC

Invoked by candidates to gather votes (§5.2).

Arguments:

term	candidate's term
candidateId	candidate requesting vote
lastLogIndex	index of candidate's last log entry (§5.4)
lastLogTerm	term of candidate's last log entry (§5.4)

Results:

term	currentTerm, for candidate to update itself
voteGranted	true means candidate received vote

Receiver implementation:

1. Reply false if term < currentTerm (§5.1)
2. If votedFor is null or candidateId, and candidate's log is at least as up-to-date as receiver's log, grant vote (§5.2, §5.4)

Rules for Servers

All Servers:

- If commitIndex > lastApplied: increment lastApplied, apply log[lastApplied] to state machine (§5.3)
- If RPC request or response contains term T > currentTerm: set currentTerm = T, convert to follower (§5.1)

Followers (§5.2):

- Respond to RPCs from candidates and leaders
- If election timeout elapses without receiving AppendEntries RPC from current leader or granting vote to candidate: convert to candidate

Candidates (§5.2):

- On conversion to candidate, start election:
 - Increment currentTerm
 - Vote for self
 - Reset election timer
 - Send RequestVote RPCs to all other servers
- If votes received from majority of servers: become leader
- If AppendEntries RPC received from new leader: convert to follower
- If election timeout elapses: start new election

Leaders:

- Upon election: send initial empty AppendEntries RPCs (heartbeat) to each server; repeat during idle periods to prevent election timeouts (§5.2)
- If command received from client: append entry to local log, respond after entry applied to state machine (§5.3)
- If last log index $\geq$ nextIndex for a follower: send AppendEntries RPC with log entries starting at nextIndex
 - If successful: update nextIndex and matchIndex for follower (§5.3)
 - If AppendEntries fails because of log inconsistency: decrement nextIndex and retry (§5.3)
- If there exists an N such that N > commitIndex, a majority of matchIndex[i] $\geq$ N, and log[N].term == currentTerm: set commitIndex = N (§5.3, §5.4).

实验记录



实验要求

- ☐ raft.go Raft 结构体、Make、Start、GetState、持久化相关接口、AppendEntries / RequestVote 等 RPC 与内部逻辑。状态机、选举、日志、提交、ApplyMsg 等都在这里完成。
- ☐ config.go 搭 labrpc 网络、创建多个 Raft、连/断节点、统计 RPC、从 applyCh 读提交结果、检查是否只有一个 leader 等。测试依赖它，一般不用改逻辑。
- ☐ test_test.go 2A（选举）、2B（日志）、2C（持久化）、2D（快照）等 Test*，调用 config.go 里的辅助函数。
- ☐ persister.go (内存模拟磁盘)：保存 raftstate 和 (2D) snapshot，提供 SaveRaftState / ReadRaftState / Snapshot 等 API。Make 里会注入 *Persister，2C/2D 要通过它读写状态。
- ☐ util.go DPrintf + Debug 开关，需要时在 raft.go 里打日志。

实现细节

2A: Leader Election（领导者选举）

在A部分，我们要保证在任何任期（Term）内最多只有一个 Leader，并处理节点状态（Follower, Candidate, Leader）的转换。

- **心跳与选举定时器 (ticker)**: 代码启动了 ticker 协程，使用 select 监听 electionTimer 和 heartbeatTimer。当 electionTimer 超时且当前不是 Leader 时，触发 startElection()。
- **发起选举 (startElection)**: 节点自增 currentTerm，转为 Candidate，先投自己一票，并重置随机的选举超时时间（300ms-500ms，避免脑裂）。随后并发地向其他节点发送 RequestVote RPC。
- **投票逻辑 (RequestVote)**: 节点在收到拉票请求时，会进行严格的检查。首先拒绝旧任期的请求；如果发现更高任期，就更新任期并立刻降级为 Follower。最关键的是**选举限制 (Election Restriction)**，通过 isMoreUpToDate() 函数，确保只有包含自己所有已提交日志的 Candidate 才能赢得选举（比较最后一条日志的 Term 和 Index）。
- **当选 Leader (becomeLeader)**: 当收到半数以上（Majority）的赞成票时，节点转为 Leader，初始化 nextIndex（所有节点设为当前日志长度加一）和 matchIndex（全为 0），并立即触发一次心跳以建立权威。初始化 nextIndex 和 matchIndex 是基于乐观估计所有 follower 都和 leader 一样新，如果错了就快速回退。matchIndex 初始化是为了重新同步，leader 还不知道 follower 现在又哪些日志

2B: Log Replication（日志复制）

这部分确保 Leader 收到的指令能够一致地复制到多数 Follower 的状态机中。

- **接收客户端指令 (Start)**: 当上层调用 `Start()` 时, Leader 会将新命令封装成 `LogEntry` 追加到本地日志中, 更新自身的 `matchIndex`, 并立即重置心跳定时器 (`rf.heartbeatTimer.Reset(0)`), 从而瞬间触发对所有 Follower 的日志复制 (`replicateTo`)。
- **发送日志 (replicateTo)**: Leader 为每个 Follower 组装 `AppendEntriesArgs`, 其中包含 `PrevLogIndex` 和 `PrevLogTerm` 用于一致性检查。
- **一致性检查与追加 (AppendEntries)**: Follower 收到日志后, 有三次检测。第一次先检测 `PrevLogIndex` 是否在节点的快照中, 如果在就返回 `reply.ConflictIndex = rf.lastIncludedIndex + 1`, 然后返回主节点, 让主节点更新 `nextIndex`。第二次先检测 `PrevLogIndex` 是否超过了节点最大日志长度, 如果超过了就返回 `reply.ConflictIndex = rf.lastLogIndex() + 1`, 然后返回主节点, 让主节点更新 `nextIndex`。第三次检查对比 `PrevLogIndex` 处的日志 Term 是否匹配。在这里我们实现了**快速回退优化 (Fast Rollback)**: 如果不匹配, Follower 会返回冲突任期的前一个任期 (`ConflictTerm`) 及其最后一条日志的索引 (`ConflictIndex`), 这使得 Leader 可以一次跨越一个任期来回退 `nextIndex`, 大大加快了网络不稳定时的恢复速度。匹配成功后, Follower 截断冲突日志并追加新日志, 然后持久化核心状态, 并更新 `commitIndex`, 通道传递信息给上层状态机执行新提交的日志。
- **推进 Commit Index (tryCommit)**: Leader 收到复制成功的回复后, 更新对应节点的 `matchIndex`。随后在 `tryCommit()` 中寻找多数节点都已复制、且属于当前任期的最大 `Index`, 将其更新为 `commitIndex`, 并调用 `rf.applyCond.Signal()` 唤醒 Apply 协程。
- **异步应用状态机 (applier)**: 独立的 `applier` 协程在 `rf.applyCond.Wait()` 处阻塞。一旦 `commitIndex > lastApplied`, 它就会醒来, 将这部分确认提交的日志打包成 `ApplyMsg` 发送入 `applyCh`, 供上层状态机执行。

2C: Persistence (持久化)

这部分为了保证节点崩溃重启后能够恢复状态, 避免破坏 Raft 的一致性保证。

- **持久化时机**: 只要 `currentTerm`、`votedFor` 或 `log` 发生改变, 代码就会调用 `rf.persist()`。
- **保存的数据 (persist / readPersist)**: 使用 `labgob` 序列化五个核心状态。除了 Raft 论文规定的 `currentTerm`、`votedFor` 和 `log` 之外, 因为引入了 2D 的快照机制, 还必须一并持久化 `lastIncludedIndex` 和 `lastIncludedTerm`, 以保证日志索引的连续性。

2D: Log Compaction (日志压缩与快照)

随着系统运行, 日志会越来越长。这部分负责截断旧日志, 并通过快照同步落后的 Follower。

- **虚拟索引转换 (`toSliceIndex`)**: 这因为日志切片不再包含从 0 开始的所有日志，封装 `toSliceIndex`、`lastLogIndex` 和 `termAt` 工具函数，在全局逻辑 Index 和 Go 语言切片的物理之间做无缝转换。
- **主动打快照 (`Snapshot` 压缩日志)**: 上层状态机检测到日志过大时，会调用此方法。Raft 将日志数据中指定 `index` 及其之前的日志全部丢弃（直接截断日志切片），如果 `index <= lastIncludedIndex` 就会直接返回，因为新快照还没有旧快照新，否则就会更新 `lastIncludedIndex` 和 `lastIncludedTerm`，并将新的状态（`currentTerm`、`votedFor`、`log`、`lastIncludedIndex`、`lastIncludedTerm` 其中 `log` 是截断后剩下的日志数组数据）和快照数据（状态机的 KV 数据）原子性地保存到持久化存储中（`SaveStateAndSnapshot`）。
- **发送快照 (`replicateTo` 分支被动同步节点状态)**: 在 Leader 向 Follower 发送日志时，如果发现 Follower 需要的日志 `nextIdx` 已经被 Leader 快照截断（`nextIdx <= rf.lastIncludedIndex`），Leader 会自动放弃发送 `AppendEntries`，转而发送 `InstallSnapshot` RPC。
- **接收快照 (`InstallSnapshot` RPC 处理函数（同步快照数据给状态机） & `CondInstallSnapshot` 快照保存函数)**: Follower 收到 Leader 的快照后，判断快照的新旧程度，如果传过来的快照更新的话，将其通过 `applyCh` 传递给上层。上层状态机加载完快照后，会回调 `CondInstallSnapshot` 将快照数据持久化到 follower 节点本地。在这里，Follower 判断快照是否确实比自己当前的提交进度新，如果新，则清空/截断本地的陈旧日志，同步更新快照元数据，并完成持久化。

raft.go

```

1  package raft
2
3  import (
4      "6.824/labgob"
5      "6.824/labrpc"
6      "bytes"
7      "math/rand"
8      "sync"
9      "sync/atomic"
10     "time"
11 )
12
13 const (
14     StateFollower = iota
15     StateCandidate
16     StateLeader
17 )
18
19 const (
```

```

20     HeartbeatInterval = 100 * time.Millisecond
21     ElectionTimeoutMin = 300
22     ElectionTimeoutMax = 500
23 )
24
25 // raft -> stateMachine
26 type ApplyMsg struct {
27     CommandValid bool
28     Command      interface{}
29     CommandIndex int
30
31     SnapshotValid bool
32     Snapshot      []byte
33     SnapshotTerm  int
34     SnapshotIndex int
35 }
36
37 type LogEntry struct {
38     Term      int
39     Command   interface{}
40 }
41
42 type Raft struct {
43     mu          sync.Mutex // 互斥锁，保护共享数据的一致性
44     peers       []*labrpc.ClientEnd // 集群中所有节点的RPC客户端端点
45     persister   *Persister          // 持久化存储器，用于保存Raft状态
46     me          int                 // 当前节点在集群中的编号
47     dead        int32              // 标记节点是否死亡
48     state       int                 // 当前节点状态: Follower/Candidate/Leader
49
50     applyCh chan ApplyMsg // 通道，用于向上层应用传递已提交的日志命令
51
52     // 持久化状态（需要写入磁盘）
53     currentTerm int // 节点当前的任期号
54     votedFor    int // 当前任期投给的候选人ID（-1表示未投票）
55     log         []LogEntry // 日志条目集合
56
57     // 优化的持久化状态（用于快照）
58     lastIncludedIndex int // 快照中包含的最后日志索引
59     lastIncludedTerm  int // 快照中包含的最后日志任期
60
61     // 易失状态（重启后可重新计算）
62     commitIndex int // 已知被提交的最高日志索引
63     lastApplied int // 已应用到状态机的最高日志索引
64
65     // Leader专用的易失状态
66     nextIndex []int // 记录发送给每个follower的下一个日志索引

```

```

67     matchIndex []int // 记录每个follower已复制的最高日志索引
68
69     // 定时器
70     electionTimer *time.Timer // 选举超时定时器
71     heartbeatTimer *time.Timer // 心跳发送定时器
72
73     // 条件变量
74     applyCond *sync.Cond // 用于触发日志应用
75 }
76
77 // ===== 工具函数 =====
78
79 func randomElectionTimeout() time.Duration {
80     ms := ElectionTimeoutMin + rand.Intn(ElectionTimeoutMax-ElectionTimeoutMin)
81     return time.Duration(ms) * time.Millisecond
82 }
83
84 // 将全局 index 转为 rf.log 中的切片下标
85 func (rf *Raft) toSliceIndex(globalIndex int) int {
86     return globalIndex - rf.lastIncludedIndex - 1
87 }
88
89 // 获取 rf.log 中最后一条日志的全局 index
90 func (rf *Raft) lastLogIndex() int {
91     return rf.lastIncludedIndex + len(rf.log)
92 }
93
94 // 获取 rf.log 中最后一条日志的任期
95 func (rf *Raft) lastLogTerm() int {
96     if len(rf.log) == 0 {
97         return rf.lastIncludedTerm
98     }
99     return rf.log[len(rf.log)-1].Term
100 }
101
102 // 获取指定全局 index 的日志任期
103 func (rf *Raft) termAt(globalIndex int) int {
104     if globalIndex == rf.lastIncludedIndex {
105         return rf.lastIncludedTerm
106     }
107     if globalIndex < rf.lastIncludedIndex {
108         return -1
109     }
110     sliceIdx := rf.toSliceIndex(globalIndex)
111     if sliceIdx < 0 || sliceIdx >= len(rf.log) {
112         return -1
113     }

```

```
114     return rf.log[sliceIdx].Term
115 }
116
117 // ===== 状态获取 =====
118
119 func (rf *Raft) GetState() (int, bool) {
120     rf.mu.Lock()
121     defer rf.mu.Unlock()
122     return rf.currentTerm, rf.state == StateLeader
123 }
124
125 // ===== 2C 持久化 =====
126
127 func (rf *Raft) persist() {
128     w := new(bytes.Buffer)
129     e := labgob.NewEncoder(w)
130     e.Encode(rf.currentTerm)
131     e.Encode(rf.votedFor)
132     e.Encode(rf.log)
133     e.Encode(rf.lastIncludedIndex)
134     e.Encode(rf.lastIncludedTerm)
135     data := w.Bytes()
136     rf.persister.SaveRaftState(data)
137 }
138
139 func (rf *Raft) readPersist(data []byte) {
140     if data == nil || len(data) < 1 {
141         return
142     }
143     r := bytes.NewBuffer(data)
144     d := labgob.NewDecoder(r)
145     var currentTerm int
146     var votedFor int
147     var log []LogEntry
148     var lastIncludedIndex int
149     var lastIncludedTerm int
150     if d.Decode(&currentTerm) != nil ||
151         d.Decode(&votedFor) != nil ||
152         d.Decode(&log) != nil ||
153         d.Decode(&lastIncludedIndex) != nil ||
154         d.Decode(&lastIncludedTerm) != nil {
155         return
156     }
157     rf.currentTerm = currentTerm
158     rf.votedFor = votedFor
159     rf.log = log
160     rf.lastIncludedIndex = lastIncludedIndex
```

```

161     rf.lastIncludedTerm = lastIncludedTerm
162     rf.commitIndex = lastIncludedIndex
163     rf.lastApplied = lastIncludedIndex
164 }
165
166 // ===== 2D 快照 =====
167
168 func (rf *Raft) CondInstallSnapshot(lastIncludedTerm int, lastIncludedIndex
int, snapshot []byte) bool {
169     rf.mu.Lock()
170     defer rf.mu.Unlock()
171
172     // 如果快照已经过时, 拒绝
173     if lastIncludedIndex <= rf.commitIndex {
174         return false
175     }
176
177     // 截断日志
178     if lastIncludedIndex <= rf.lastLogIndex() {
179         sliceIdx := rf.toSliceIndex(lastIncludedIndex)
180         rf.log = rf.log[sliceIdx+1:]
181     } else {
182         rf.log = []LogEntry{}
183     }
184
185     rf.lastIncludedIndex = lastIncludedIndex
186     rf.lastIncludedTerm = lastIncludedTerm
187     rf.commitIndex = lastIncludedIndex
188     rf.lastApplied = lastIncludedIndex
189
190     w := new(bytes.Buffer)
191     e := labgob.NewEncoder(w)
192     e.Encode(rf.currentTerm)
193     e.Encode(rf.votedFor)
194     e.Encode(rf.log)
195     e.Encode(rf.lastIncludedIndex)
196     e.Encode(rf.lastIncludedTerm)
197     rf.persister.SaveStateAndSnapshot(w.Bytes(), snapshot)
198
199     return true
200 }
201
202 func (rf *Raft) Snapshot(index int, snapshot []byte) {
203     rf.mu.Lock()
204     defer rf.mu.Unlock()
205
206     if index <= rf.lastIncludedIndex {

```

```

207         return
208     }
209
210     sliceIdx := rf.toSliceIndex(index)
211     rf.lastIncludedTerm = rf.log[sliceIdx].Term
212     rf.log = rf.log[sliceIdx+1:]
213     rf.lastIncludedIndex = index
214
215     w := new(bytes.Buffer)
216     e := labgob.NewEncoder(w)
217     e.Encode(rf.currentTerm)
218     e.Encode(rf.votedFor)
219     e.Encode(rf.log)
220     e.Encode(rf.lastIncludedIndex)
221     e.Encode(rf.lastIncludedTerm)
222     rf.persister.SaveStateAndSnapshot(w.Bytes(), snapshot)
223 }
224
225 // ===== 2A 选举相关 =====
226
227 type RequestVoteArgs struct {
228     Term          int
229     CandidateId   int
230     LastLogIndex  int
231     LastLogTerm   int
232 }
233
234 type RequestVoteReply struct {
235     Term          int
236     VoteGranted   bool
237 }
238
239 // 日志是否比自己的更新 (up-to-date)
240 func (rf *Raft) isMoreUpToDate(lastLogIndex, lastLogTerm int) bool {
241     myLastTerm := rf.lastLogTerm()
242     myLastIndex := rf.lastLogIndex()
243     if lastLogTerm != myLastTerm {
244         return lastLogTerm > myLastTerm
245     }
246     return lastLogIndex >= myLastIndex
247 }
248
249 func (rf *Raft) RequestVote(args *RequestVoteArgs, reply *RequestVoteReply) {
250     rf.mu.Lock()
251     defer rf.mu.Unlock()
252
253     reply.Term = rf.currentTerm

```

```

254     reply.VoteGranted = false
255
256     // 拒绝旧任期的请求
257     if args.Term < rf.currentTerm {
258         return
259     }
260
261     // 发现更高任期, 退回 follower
262     if args.Term > rf.currentTerm {
263         rf.currentTerm = args.Term
264         rf.state = StateFollower
265         rf.votedFor = -1
266         rf.persist()
267     }
268
269     reply.Term = rf.currentTerm
270
271     // 检查是否可以投票 + 日志是否足够新
272     canVote := rf.votedFor == -1 || rf.votedFor == args.CandidateId
273     if canVote && rf.isMoreUpToDate(args.LastLogIndex, args.LastLogTerm) {
274         rf.votedFor = args.CandidateId
275         rf.persist()
276         reply.VoteGranted = true
277         rf.resetElectionTimer()
278     }
279 }
280
281 func (rf *Raft) sendRequestVote(server int, args *RequestVoteArgs, reply
    *RequestVoteReply) bool {
282     ok := rf.peers[server].Call("Raft.RequestVote", args, reply)
283     return ok
284 }
285
286 func (rf *Raft) startElection() {
287     rf.state = StateCandidate
288     rf.currentTerm++
289     rf.votedFor = rf.me
290     rf.persist()
291     rf.resetElectionTimer()
292
293     args := &RequestVoteArgs{
294         Term:          rf.currentTerm,
295         CandidateId:    rf.me,
296         LastLogIndex:   rf.lastLogIndex(),
297         LastLogTerm:    rf.lastLogTerm(),
298     }
299

```

```

300     votes := 1
301     majority := len(rf.peers)/2 + 1
302
303     for peer := range rf.peers {
304         if peer == rf.me {
305             continue
306         }
307         go func(server int) {
308             reply := &RequestVoteReply{}
309             if !rf.sendRequestVote(server, args, reply) {
310                 return
311             }
312             rf.mu.Lock()
313             defer rf.mu.Unlock()
314
315             if reply.Term > rf.currentTerm {
316                 rf.currentTerm = reply.Term
317                 rf.state = StateFollower
318                 rf.votedFor = -1
319                 rf.persist()
320                 return
321             }
322
323             // 确认仍处于同一任期的 Candidate
324             if rf.state != StateCandidate || rf.currentTerm != args.Term {
325                 return
326             }
327
328             if reply.VoteGranted {
329                 votes++
330                 if votes >= majority {
331                     rf.becomeLeader()
332                 }
333             }
334         }(peer)
335     }
336 }
337
338 func (rf *Raft) becomeLeader() {
339     rf.state = StateLeader
340     // 初始化 nextIndex 和 matchIndex
341     for i := range rf.peers {
342         rf.nextIndex[i] = rf.lastLogIndex() + 1
343         rf.matchIndex[i] = 0
344     }
345     rf.matchIndex[rf.me] = rf.lastLogIndex()
346     // 立即发送心跳

```

```

347     rf.heartbeatTimer.Reset(0)
348 }
349
350 func (rf *Raft) resetElectionTimer() {
351     rf.electionTimer.Reset(randomElectionTimeout())
352 }
353
354 // ===== 2B AppendEntries =====
355
356 type AppendEntriesArgs struct {
357     Term          int          // Leader的当前任期号
358     LeaderId       int          // Leader的节点ID
359     PrevLogIndex   int          // 新日志条目前一个日志的索引
360     PrevLogTerm    int          // 新日志条目前一个日志的任期号
361     Entries        []LogEntry  // 要追加的日志条目 (心跳时为空)
362     LeaderCommit   int          // Leader的已提交索引
363 }
364
365 type AppendEntriesReply struct {
366     Term          int
367     Success       bool
368     // 快速回退优化字段
369     ConflictTerm  int
370     ConflictIndex int
371 }
372
373 func (rf *Raft) AppendEntries(args *AppendEntriesArgs, reply
374 *AppendEntriesReply) {
375     rf.mu.Lock()
376     defer rf.mu.Unlock()
377
378     reply.Term = rf.currentTerm
379     reply.Success = false
380
381     if args.Term < rf.currentTerm {
382         return
383     }
384
385     // 发现 leader, 重置状态
386     if args.Term > rf.currentTerm {
387         rf.currentTerm = args.Term
388         rf.votedFor = -1
389         rf.persist()
390     }
391     rf.state = StateFollower
392     rf.resetElectionTimer()
393     reply.Term = rf.currentTerm

```

```

393
394 // ----- 一致性检查 -----
395 // prevLogIndex 在快照之前
396 if args.PrevLogIndex < rf.lastIncludedIndex {
397     reply.Success = false
398     reply.ConflictIndex = rf.lastIncludedIndex + 1
399     reply.ConflictTerm = -1
400     return
401 }
402
403 // prevLogIndex 超过当前日志长度
404 if args.PrevLogIndex > rf.lastLogIndex() {
405     reply.ConflictIndex = rf.lastLogIndex() + 1
406     reply.ConflictTerm = -1
407     return
408 }
409
410 // prevLog 任期不匹配
411 if rf.termAt(args.PrevLogIndex) != args.PrevLogTerm {
412     conflictTerm := rf.termAt(args.PrevLogIndex)
413     reply.ConflictTerm = conflictTerm
414     // 找到该任期的第一个 index
415     idx := args.PrevLogIndex
416     for idx > rf.lastIncludedIndex && rf.termAt(idx-1) == conflictTerm {
417         idx--
418     }
419     reply.ConflictIndex = idx
420     return
421 }
422
423 // ----- 追加日志 -----
424 for i, entry := range args.Entries {
425     globalIdx := args.PrevLogIndex + 1 + i
426     if globalIdx <= rf.lastIncludedIndex {
427         continue
428     }
429     sliceIdx := rf.toSliceIndex(globalIdx)
430     if sliceIdx < len(rf.log) {
431         // 冲突则截断
432         if rf.log[sliceIdx].Term != entry.Term {
433             rf.log = rf.log[:sliceIdx]
434             rf.log = append(rf.log, args.Entries[i:]...)
435             break
436         }
437     } else {
438         rf.log = append(rf.log, args.Entries[i:]...)
439         break

```

```

440     }
441 }
442 rf.persist()
443 reply.Success = true
444
445 // ---- 更新 commitIndex ----
446 if args.LeaderCommit > rf.commitIndex {
447     newCommit := args.LeaderCommit
448     if rf.lastLogIndex() < newCommit {
449         newCommit = rf.lastLogIndex()
450     }
451     rf.commitIndex = newCommit
452     rf.applyCond.Signal()
453 }
454 }
455
456 func (rf *Raft) sendAppendEntries(server int, args *AppendEntriesArgs, reply
*AppendEntriesReply) bool {
457     ok := rf.peers[server].Call("Raft.AppendEntries", args, reply)
458     return ok
459 }
460
461 // ===== 2D InstallSnapshot =====
462
463 type InstallSnapshotArgs struct {
464     Term          int
465     LeaderId      int
466     LastIncludedIndex int
467     LastIncludedTerm int
468     Data          []byte
469 }
470
471 type InstallSnapshotReply struct {
472     Term int
473 }
474
475 func (rf *Raft) InstallSnapshot(args *InstallSnapshotArgs, reply
*InstallSnapshotReply) {
476     rf.mu.Lock()
477     defer rf.mu.Unlock()
478
479     reply.Term = rf.currentTerm
480
481     if args.Term < rf.currentTerm {
482         return
483     }
484

```

```

485     if args.Term > rf.currentTerm {
486         rf.currentTerm = args.Term
487         rf.votedFor = -1
488         rf.persist()
489     }
490     rf.state = StateFollower
491     rf.resetElectionTimer()
492     reply.Term = rf.currentTerm
493
494     if args.LastIncludedIndex <= rf.lastIncludedIndex {
495         return
496     }
497
498     // 通过 applyCh 发送快照给上层应用
499     go func() {
500         rf.applyCh <- ApplyMsg{
501             SnapshotValid: true,
502             Snapshot:      args.Data,
503             SnapshotTerm:  args.LastIncludedTerm,
504             SnapshotIndex: args.LastIncludedIndex,
505         }
506     }()
507 }
508
509 func (rf *Raft) sendInstallSnapshot(server int, args *InstallSnapshotArgs,
reply *InstallSnapshotReply) bool {
510     ok := rf.peers[server].Call("Raft.InstallSnapshot", args, reply)
511     return ok
512 }
513
514 // ===== 2B Leader 发送日志 =====
515 // 心跳发送
516 func (rf *Raft) replicateTo(peer int) {
517     rf.mu.Lock()
518
519     if rf.state != StateLeader {
520         rf.mu.Unlock()
521         return
522     }
523     // 获取该节点需要的下一条日志的索引
524     nextIdx := rf.nextIndex[peer]
525
526     // 如果 follower 需要的日志已经在快照里，发送快照
527     if nextIdx <= rf.lastIncludedIndex {
528         args := &InstallSnapshotArgs{
529             Term:          rf.currentTerm,
530             LeaderId:       rf.me,

```

```

531         LastIncludedIndex: rf.lastIncludedIndex,
532         LastIncludedTerm:  rf.lastIncludedTerm,
533         Data:               rf.persister.ReadSnapshot(),
534     }
535     rf.mu.Unlock()
536
537     reply := &InstallSnapshotReply{}
538     if !rf.sendInstallSnapshot(peer, args, reply) {
539         return
540     }
541
542     rf.mu.Lock()
543     defer rf.mu.Unlock()
544     if reply.Term > rf.currentTerm {
545         rf.currentTerm = reply.Term
546         rf.state = StateFollower
547         rf.votedFor = -1
548         rf.persist()
549         return
550     }
551     if rf.state != StateLeader || rf.currentTerm != args.Term {
552         return
553     }
554     if args.LastIncludedIndex+1 > rf.nextIndex[peer] {
555         rf.nextIndex[peer] = args.LastIncludedIndex + 1
556     }
557     if args.LastIncludedIndex > rf.matchIndex[peer] {
558         rf.matchIndex[peer] = args.LastIncludedIndex
559     }
560     return
561 }
562 // 普通日志
563 prevLogIndex := nextIdx - 1
564 prevLogTerm := rf.termAt(prevLogIndex)
565
566 // 复制 nextIdx 之后的所有日志
567 entries := make([]LogEntry, 0)
568 if nextIdx <= rf.lastLogIndex() {
569     sliceStart := rf.toSliceIndex(nextIdx)
570     entries = append(entries, rf.log[sliceStart:]...)
571 }
572
573 args := &AppendEntriesArgs{
574     Term:         rf.currentTerm,
575     LeaderId:     rf.me,
576     PrevLogIndex: prevLogIndex,
577     PrevLogTerm:  prevLogTerm,

```

```
578     Entries:      entries,
579     LeaderCommit: rf.commitIndex,
580 }
581 rf.mu.Unlock()
582
583 reply := &AppendEntriesReply{}
584 if !rf.sendAppendEntries(peer, args, reply) {
585     return
586 }
587
588 rf.mu.Lock()
589 defer rf.mu.Unlock()
590
591 if reply.Term > rf.currentTerm {
592     rf.currentTerm = reply.Term
593     rf.state = StateFollower
594     rf.votedFor = -1
595     rf.persist()
596     return
597 }
598
599 if rf.state != StateLeader || rf.currentTerm != args.Term {
600     return
601 }
602
603 if reply.Success {
604     newMatch := args.PrevLogIndex + len(args.Entries)
605     if newMatch > rf.matchIndex[peer] {
606         rf.matchIndex[peer] = newMatch
607     }
608     rf.nextIndex[peer] = rf.matchIndex[peer] + 1
609     rf.tryCommit()
610 } else {
611     // 快速回退
612     if reply.ConflictTerm == -1 {
613         rf.nextIndex[peer] = reply.ConflictIndex
614     } else {
615         // 找到 leader 中 ConflictTerm 最后一条日志
616         newNext := reply.ConflictIndex
617         for i := rf.lastLogIndex(); i > rf.lastIncludedIndex; i-- {
618             if rf.termAt(i) == reply.ConflictTerm {
619                 newNext = i + 1
620                 break
621             }
622         }
623         rf.nextIndex[peer] = newNext
624     }
625 }
```

```

625         if rf.nextIndex[peer] < 1 {
626             rf.nextIndex[peer] = 1
627         }
628     }
629 }
630
631 // leader 尝试推进 commitIndex
632 func (rf *Raft) tryCommit() {
633     for n := rf.lastLogIndex(); n > rf.commitIndex; n-- {
634         if rf.termAt(n) != rf.currentTerm {
635             continue
636         }
637         count := 1
638         for peer := range rf.peers {
639             if peer != rf.me && rf.matchIndex[peer] >= n {
640                 count++
641             }
642         }
643         if count > len(rf.peers)/2 {
644             rf.commitIndex = n
645             rf.applyCond.Signal()
646             break
647         }
648     }
649 }
650
651 // ===== 2B Start =====
652
653 func (rf *Raft) Start(command interface{}) (int, int, bool) {
654     rf.mu.Lock()
655     defer rf.mu.Unlock()
656
657     if rf.state != StateLeader {
658         return -1, -1, false
659     }
660
661     entry := LogEntry{
662         Term:    rf.currentTerm,
663         Command: command,
664     }
665     rf.log = append(rf.log, entry)
666     rf.persist()
667
668     index := rf.lastLogIndex()
669     term := rf.currentTerm
670     rf.matchIndex[rf.me] = index
671

```

```

672 // 立即触发复制
673 rf.heartbeatTimer.Reset(0)
674
675 return index, term, true
676 }
677
678 // ===== Apply 协程 =====
679
680 func (rf *Raft) applier() {
681     for !rf.killed() {
682         rf.mu.Lock()
683         for rf.lastApplied >= rf.commitIndex {
684             rf.applyCond.Wait()
685         }
686
687         commitIndex := rf.commitIndex
688         lastApplied := rf.lastApplied
689
690         // 收集需要提交的日志
691         entries := make([]LogEntry, 0)
692         startIdx := lastApplied + 1
693         for i := startIdx; i <= commitIndex; i++ {
694             if i <= rf.lastIncludedIndex {
695                 continue
696             }
697             si := rf.toSliceIndex(i)
698             if si < 0 || si >= len(rf.log) {
699                 break
700             }
701             entries = append(entries, rf.log[si])
702         }
703         rf.mu.Unlock()
704
705         for i, entry := range entries {
706             globalIdx := startIdx + i
707             if globalIdx <= rf.lastIncludedIndex {
708                 continue
709             }
710             rf.applyCh <- ApplyMsg{
711                 CommandValid: true,
712                 Command:         entry.Command,
713                 CommandIndex:    globalIdx,
714             }
715         }
716
717         rf.mu.Lock()
718         if commitIndex > rf.lastApplied {

```

```

719         rf.lastApplied = commitIndex
720     }
721     rf.mu.Unlock()
722 }
723 }
724
725 // ===== ticker & heartbeat =====
726
727 func (rf *Raft) Kill() {
728     atomic.StoreInt32(&rf.dead, 1)
729 }
730
731 func (rf *Raft) killed() bool {
732     z := atomic.LoadInt32(&rf.dead)
733     return z == 1
734 }
735
736 func (rf *Raft) ticker() {
737     for !rf.killed() {
738         select {
739         case <-rf.electionTimer.C:
740             rf.mu.Lock()
741             if rf.state != StateLeader {
742                 rf.startElection()
743             }
744             rf.mu.Unlock()
745
746         case <-rf.heartbeatTimer.C:
747             rf.mu.Lock()
748             if rf.state == StateLeader {
749                 // 向所有 follower 发送心跳/日志
750                 for peer := range rf.peers {
751                     if peer == rf.me {
752                         continue
753                     }
754                     go rf.replicateTo(peer)
755                 }
756                 rf.heartbeatTimer.Reset(HeartbeatInterval)
757             }
758             rf.mu.Unlock()
759         }
760     }
761 }
762
763 // ===== Make =====
764
765 func Make(peers []*labrpc.ClientEnd, me int,

```

```
766     persister *Persister, applyCh chan ApplyMsg) *Raft {
767
768     rf := &Raft{}
769     rf.peers = peers
770     rf.persister = persister
771     rf.me = me
772     rf.applyCh = applyCh
773
774     rf.state = StateFollower
775     rf.currentTerm = 0
776     rf.votedFor = -1
777     rf.log = make([]LogEntry, 0)
778
779     rf.lastIncludedIndex = 0
780     rf.lastIncludedTerm = 0
781     rf.commitIndex = 0
782     rf.lastApplied = 0
783
784     rf.nextIndex = make([]int, len(peers))
785     rf.matchIndex = make([]int, len(peers))
786
787     rf.applyCond = sync.NewCond(&rf.mu)
788
789     rf.electionTimer = time.NewTimer(randomElectionTimeout())
790     rf.heartbeatTimer = time.NewTimer(HeartbeatInterval)
791
792     // 从持久化状态恢复
793     rf.readPersist(persister.ReadRaftState())
794
795     // 启动 ticker 协程
796     go rf.ticker()
797     // 启动 apply 协程
798     go rf.applier()
799
800     return rf
801 }
```